

[项目实战]广告数仓

项目简介

广告数仓项目旨在设计一套广告数仓，以帮助广告主评估广告投放效果，并识别投放期间的异常流量。

这个项目涉及到的知识点包括：

- ETL
- 建模理论
- 数据可视化

涉及到的具体技术栈包括：

- 日志增量采集
- DataX 数据全量采集
- shell 调度脚本
- hive 自定义函数
- 编写 SparkSql 将 hive 数据批量导出到 clickhouse

请想一想，在这个项目中，我们能让 AI 为我们做什么。

练习题

习题一 编写用于 ETL 的 Hive SQL

在广告数仓中，我们会在 ods 层建一张增量采集的日志表，表结构如下。

```

create external table if not exists ods_ad_log_inc
(
    time_local STRING comment '日志服务器收到的请求的时间',
    request_method STRING comment 'HTTP 请求方法',
    request_uri STRING comment '请求路径',
    status STRING comment '日志服务器相应状态',
    server_addr STRING comment '日志服务器自身 ip'
) PARTITIONED BY (`dt` STRING)
    row format delimited fields terminated by '\u0001'
    LOCATION '/warehouse/ad/ods/ods_ad_log_inc';

```

这张表里，每新增一条记录都意味着我们有一条广告被展示或者点击了。具体的广告展示信息，都在 request_uri 字段里。

下面就是一个 request_uri 记录：

```
"/ad/baidu/impression?id=89&t=1676394705587&ip=122.189.79.23&ua=Mozilla/5.0%20(Win
dows%20NT%2010.0)%20AppleWebKit/537.36%20(KHTML,%20like%20Gecko)%20Chrome/40.0.221
4.93%20Safari/537.36&device_id= d4b8f3898515056278ccf78a7a2cca2d&os_type=Android"
```

可以看到，标红的路径参数和查询参数分别包含了如下信息：

event_time (参数 t)

event_type (路径中的第三级)

ad_id (参数 id)

platform (路径中的第二级)

client_ip (参数 ip)

client_ua (参数 ua)

client_os_type (参数 os)

client_device_id (参数 device_id)

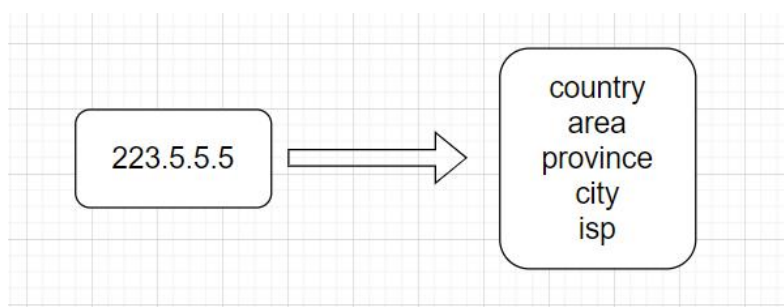
现在，为了便于分析，我们需要将 uri 里的信息提取成多个字段。现在，我们就可以让 GPT 来帮我们写这个 sql。

习题二 编写解析 User-agent 的 Hive UDF

User-agent 是 Http 协议中，广泛使用的一个介绍请求放身份角色的请求头字段。下面就是一个 User-agent

习题三 编写解析 ipv4 地址的 Hive UDF（GPT 做不到）

本项目中有一个非常重要的需求就是将 ipv4 地址解析为地理信息。



要实现这个功能，我们需要想办法拿到能讲 ip 映射地理信息的数据或者服务。这个时候一般有三种方案：

1. **调用接口：**很多云服务商都会提供 ip-地理信息的查询接口，这种接口往往物美价廉。但是在大数据开发的场景中我们通常不会使用调用接口的方式，这是因为接口调用涉及 http 的请求与响应，这是一种比较消耗资源的方式，而且一般接口供应方都会做接口调用速率的限制。
2. **使用商用 ip 地理信息离线数据：**这也是一种常用的方法，有的云服务提供商会允许你把 ip 地理信息数据下载到本地，然后自己使用这些数据去做搜索功能。在大数据场景下，应尽可能地选择这类离线搜索的实现方式。
3. **开源 ip 地理信息离线数据：**跟商业版的离线数据一样，我们也可以去 github 上找开源的 ip 地理信息离线数据，这种方式拿到的数据相对来说要不如商用 ip 地理解析服务精准，但也在可接受的范围之内。

ip2region 就是一个在 github 上开源的 ip 地理信息离线数据库，不过在一年前，**ip2region** 社区为了加快 ip 检索的速度，减少计算压力，修改了 ip 地理数据的存储方式——xdb。这个 xdb 的查询性能上快准狠，但是查询的时候得让客户端支持 xdb 的协议，后来 **ip2region** 社区又推出了各个编程语言里的 **xdb** 查询客户端。但是这都晚于 21 年 9 月，所以我们无法让 ChatGPT 写出这一领域的可用代码。

参考答案

习题一 编写用于 ETL 的 Hive SQL

第 1 轮提示

提示

我有一张 hive 的表 ods_ad_log_inc。它的 ddl 如下

```
"""create external table if not exists ods_ad_log_inc
(
    time_local STRING comment '日志服务器收到的请求的时间',
    request_method STRING comment 'HTTP 请求方法',
    request_uri STRING comment '请求路径',
    status STRING comment '日志服务器相应状态',
    server_addr STRING comment '日志服务器自身 ip'
) PARTITIONED BY (`dt` STRING)
row format delimited fields terminated by '\u0001'
LOCATION '/warehouse/ad/ods/ods_ad_log_inc';
"""
```

其中，request_uri 字段里的每一行数据格式如下：

```
"/ad/baidu/impression?id=89&t=1676394705587&ip=122.189.79.23&ua=Mozilla/5.0%20(Win  
dows%20NT%2010.0)%20AppleWebKit/537.36%20(KHTML,%20like%20Gecko)%20Chrome/40.0.221  
4.93%20Safari/537.36&device_id= d4b8f3898515056278ccf78a7a2cca2d&os_type=Android"
```

我想写一个 sql 查询查出如下这些字段

event_time (参数 t)

event_type (路径中的第三级)

ad_id (参数 id)

platform (路径中的第二级)

client_ip (参数 ip)

client_ua (参数 ua)

client_os_type (参数 os)

client_device_id (参数 device_id) 另外，ods_ad_log_inc 表有一个分区字段 dt (年-月-日)，请用分区进行过滤

答

根据您的需求，可以使用以下 Hive SQL 查询来获取所需的字段：

```
WITH parsed AS (  
    SELECT  
        dt,  
        time_local,  
        regexp_extract(request_uri, '\/(\\w+)\/(\\w+)\/', 2) AS platform,  
        regexp_extract(request_uri, '\/(\\w+)\/(\\w+)\/', 3) AS event_type,  
        regexp_extract(request_uri, 'id=(\\d+)', 1) AS ad_id,  
        regexp_extract(request_uri, 't=(\\d+)', 1) AS event_time,  
        regexp_extract(request_uri, 'ip=(\\d\\.\\d\\.\\d\\.\\d+)', 1) AS client_ip,  
        regexp_extract(request_uri, 'ua=([^&]+)', 1) AS client_ua,
```

```

        regexp_extract(request_uri, 'device_id=([^&]+)', 1) AS client_device_id,
        regexp_extract(request_uri, 'os_type=([^&]+)', 1) AS client_os_type
FROM
    ods_ad_log_inc
WHERE
    dt = 'your_partition_date' -- 替换为您需要查询的日期，例如 '2023-04-26'
)
SELECT
    dt,
    time_local,
    platform,
    event_type,
    ad_id,
    event_time,
    client_ip,
    client_ua,
    client_device_id,
    client_os_type
FROM
    parsed;

```

请注意，您需要将 'your_partition_date' 替换为您需要查询的日期，例如 '2023-04-26'。这个查询会提取出您所需要的字段，并对数据进行解析。

反馈

GPT 使用了正则方案来匹配 uri 中的内容，也是一个可行的办法，下面是我们项目课程中给出的 SQL。

```

select
    parse_url('http://www.example.com' || request_uri, 'QUERY', 't') event_time,
    split(parse_url('http://www.example.com' || request_uri, 'PATH'), '/') [3] event_type,
    parse_url('http://www.example.com' || request_uri, 'QUERY', 'id') ad_id,
    split(parse_url('http://www.example.com' || request_uri, 'PATH'), '/') [2] platform,

```

```
parse_url('http://www.example.com' || request_uri, 'QUERY', 'ip') client_ip,
reflect('java.net.URLDecoder', 'decode',
parse_url('http://www.example.com' || request_uri, 'QUERY', 'ua'), 'utf-8') client_ua,
parse_url('http://www.example.com' || request_uri, 'QUERY', 'os_type') client_os_type,
parse_url('http://www.example.com' || request_uri, 'QUERY', 'device_id') client_device_id
from ods_ad_log_inc
where dt='2023-01-07';
```

注意，本轮提示的时候我们漏了一个需求，那就是 ua 的字符串本是经过 URLEncode 编码的，我们还要使用 URLDecode 进行解码。现在，也可以把这个问题交给 GPT 来做

第 2 轮提示

提示

我要增加一个需求 client_ua 的原始数据使用了 url_encode，我现在希望将它解码成正常的字符串

答

为了将 **client_ua** 反编码为正常的字符串，您需要使用 **unhex** 和 **replace** 函数。请参考以下修改后的查询：

```
WITH parsed AS (
  SELECT
    dt,
    time_local,
    regexp_extract(request_uri, '\/(\\w+)\\/(\\w+)\\/', 2) AS platform,
    regexp_extract(request_uri, '\/(\\w+)\\/(\\w+)\\/', 3) AS event_type,
    regexp_extract(request_uri, 'id=(\\d+)', 1) AS ad_id,
    regexp_extract(request_uri, 't=(\\d+)', 1) AS event_time,
    regexp_extract(request_uri, 'ip=([\\d\\.]+)', 1) AS client_ip,
    regexp_extract(request_uri, 'ua=([^&]+)', 1) AS client_ua_encoded,
    regexp_extract(request_uri, 'device_id=([^&]+)', 1) AS client_device_id,
```

```

        regexp_extract(request_uri, 'os_type=([^\&]+)', 1) AS client_os_type
FROM
    ods_ad_log_inc
WHERE
    dt = 'your_partition_date' -- 替换为您需要查询的日期，例如 '2023-04-26'
)
SELECT
    dt,
    time_local,
    platform,
    event_type,
    ad_id,
    event_time,
    client_ip,
    reflect("java.net.URLDecoder", "decode", client_ua_encoded, "UTF-8") AS client_ua,
    client_device_id,
    client_os_type
FROM
    parsed;

```

在这个查询中，我使用了 `reflect` 函数，调用 Java 的 `URLDecoder.decode` 方法将经过 URL 编码的 `client_ua_encoded` 转换为正常的字符串。请注意，您需要将 `'your_partition_date'` 替换为您需要查询的日期，例如 `'2023-04-26'`。

反馈

现在就完全正确了。

习题二 编写解析 User-agent 的 Hive UDF

第 1 轮提示

提示

我想写一个 HiveUDF

它的输入参数是一个字符串字段，里面的内容是 user agent

要求返回下面的字段

browser

browserVersion

engine

engineVersion

os

osVersion

platform

isMobile

答

要实现这个 Hive UDF，首先需要创建一个 Java 类来扩展

`org.apache.hadoop.hive.ql.exec.UDF`，然后实现一个方法来解析 User Agent。可以使用开源库 `user-agent-utils` 来简化解析过程。下面是一个示例：

1. 添加 `user-agent-utils` 依赖到项目的 `pom.xml` 文件：

```
<dependencies>
  <dependency>
```

```
<groupId>eu.bitwalker</groupId>

<artifactId>UserAgentUtils</artifactId>

<version>1.21</version>

</dependency>

</dependencies>
```

2. 创建一个 Java 类 **UserAgentUDF**，继承自 **org.apache.hadoop.hive.ql.exec.UDF**:

```
import org.apache.hadoop.hive.ql.exec.UDF;
import org.apache.hadoop.io.Text;
import eu.bitwalker.useragentutils.UserAgent;
import eu.bitwalker.useragentutils.Browser;
import eu.bitwalker.useragentutils.Version;
import eu.bitwalker.useragentutils.OperatingSystem;

public class UserAgentUDF extends UDF {

    public Text evaluate(Text userAgentString) {

        if (userAgentString == null) {

            return null;

        }

        UserAgent userAgent = UserAgent.parseUserAgentString(userAgentString.toString());
        Browser browser = userAgent.getBrowser();
        Version browserVersion = browser.getVersion(userAgentString.toString());
        OperatingSystem os = userAgent.getOperatingSystem();
        String browserName = browser.getName();
        String browserVersionNumber = browserVersion.getVersion();
        String engine = browser.getRenderingEngine().getName();
        String engineVersion = ""; // UserAgentUtils doesn't provide engine version
        String osName = os.getName();
        String osVersion = os.getVersion(userAgentString.toString());
        String platform = os.getDeviceType().getName();
        boolean isMobile = os.isMobileDevice();

        return new Text(String.format("%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s",
            browserName, browserVersionNumber, engine, engineVersion, osName, osVersion, platform, isMobile));
    }
}
```

```
        browserName,  
        browserVersionNumber,  
        engine,  
        engineVersion,  
        osName,  
        osVersion,  
        platform,  
        isMobile  
    ));  
}  
}
```

3. 编译和打包 Java 项目，将生成的 JAR 文件上传到 Hive 集群。

4. 在 Hive 中创建一个新的临时函数，并指向编译好的 JAR 文件：

```
ADD JAR /path/to/your/compiled/udf.jar;  
CREATE TEMPORARY FUNCTION parse_user_agent AS 'your.package.name.UserAgentUDF';
```

现在可以使用 **parse_user_agent** 函数解析 User Agent 字符串。例如：

```
SELECT parse_user_agent(user_agent) FROM your_table;
```

这个 UDF 将返回一个以制表符分隔的字符串，包含了浏览器、浏览器版本、引擎、引擎版本、操作系统、操作系统版本、平台和是否移动设备等信息。你可以使用 **split** 函数将它们分开，然后使用 **ARRAY** 函数来将它们提取为单独的字段。

反馈

首先，GPT 实现了 UDF 接口，UDF 接口在 hive3.1.2 里已经标记为废弃了（尚未正式废弃）。所以这个函数这样写其实也行也能用。另外，GPT 为我们选择了一个开源的工具包

UserAgentUtils

```
String browserVersionNumber = browserVersion.getVersion();
String engine = browser.getRenderingEngine().getName();
String engineVersion = ""; // UserAgentUtils doesn't provide engine version
String osName = os.getName();
```

选这个工具类正常来说没有什么问题，这个工具类也是用的比较多的。但是它有一处不满足我们的需求，就是不能从 user agent 里解析出 engineVersion。这个时候我们可以要求 GPT 换个工具类。

第 2 轮提示

提示

我想使用 hutool 来实现 user-agent 的解析

答

要使用 hutool 来实现 User-Agent 的解析，首先需要将 hutool 依赖添加到项目的 **pom.xml** 文件中：

```
<dependencies>
  <dependency>
    <groupId>cn.hutool</groupId>
    <artifactId>hutool-all</artifactId>
    <version>5.7.17</version>
  </dependency>
</dependencies>
```

接下来，创建一个 Java 类 **HutoolUserAgentUDF**，继承自

org.apache.hadoop.hive.ql.exec.UDF，并使用 hutool 的 **UserAgentUtil** 进行解析：

```
import cn.hutool.http.useragent.UserAgent;
import cn.hutool.http.useragent.UserAgentUtil;
import org.apache.hadoop.hive.ql.exec.UDF;
import org.apache.hadoop.io.Text;
```

```

public class HutoolUserAgentUDF extends UDF {
    public Text evaluate(Text userAgentString) {
        if (userAgentString == null) {
            return null;
        }

        UserAgent userAgent = UserAgentUtil.parse(userAgentString.toString());
        String browser = userAgent.getBrowser().toString();
        String browserVersion = userAgent.getVersion();
        String engine = userAgent.getEngine().toString();
        String engineVersion = userAgent.getEngineVersion();
        String os = userAgent.getOs().toString();
        String osVersion = userAgent.getOsVersion();
        String platform = userAgent.getPlatform().toString();
        boolean isMobile = userAgent.isMobile();

        return new Text(String.format("%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s",
            browser,
            browserVersion,
            engine,
            engineVersion,
            os,
            osVersion,
            platform,
            isMobile
        ));
    }
}

```

与之前的示例相似，编译并打包 Java 项目，将生成的 JAR 文件上传到 Hive 集群。

在 Hive 中创建一个新的临时函数，并指向编译好的 JAR 文件：

```

ADD JAR /path/to/your/compiled/udf.jar;
CREATE TEMPORARY FUNCTION parse_user_agent_hutool AS

```

```
'your.package.name.HutoolUserAgentUDF';
```

现在可以使用 `parse_user_agent_hutool` 函数解析 User Agent 字符串。例如：

```
SELECT parse_user_agent_hutool(user_agent) FROM your_table;
```

此 UDF 将返回一个以制表符分隔的字符串，其中包含浏览器、浏览器版本、引擎、引擎版本、操作系统、操作系统版本、平台和是否移动设备等信息。你可以使用 `split` 函数将它们分开，然后使用 `ARRAY` 函数将它们提取为单独的字段。

反馈

现在已经完全可以满足我们的要求了，唯一的美中不足的就是它返回的是一个字符串。现在我们可以让它把返回类型修改为一个 Struct。

第 3 轮提示

提示

我希望反馈的结果能是一个 struct，而不是 string

答

为了返回一个 `struct` 类型的结果，你需要更改 UDF 的返回类型以及返回值的构建方式。在这个例子中，我们将使用

`org.apache.hadoop.hive.serde2.objectinspector.ObjectInspectorFactory` 和 `org.apache.hadoop.hive.serde2.objectinspector.StructObjectInspector` 来定义返回类型，并创建一个 `ArrayList` 对象来存储返回值。

首先，修改 `HutoolUserAgentUDF` 类，覆盖 `initialize` 方法来定义返回类型，然后在 `evaluate` 方法中创建并填充一个 `ArrayList` 对象作为返回值：

```
import cn.hutool.http.useragent.UserAgent;  
import cn.hutool.http.useragent.UserAgentUtil;
```

```

import org.apache.hadoop.hive.ql.exec.UDF;
import org.apache.hadoop.hive.ql.udf.generic.GenericUDF;
import org.apache.hadoop.hive.serde2.objectinspector.ObjectInspector;
import org.apache.hadoop.hive.serde2.objectinspector.ObjectInspectorFactory;
import
org.apache.hadoop.hive.serde2.objectinspector.primitive.PrimitiveObjectInspectorFactory;
import org.apache.hadoop.hive.serde2.objectinspector.StructObjectInspector;
import org.apache.hadoop.io.Text;

import java.util.ArrayList;
import java.util.Arrays;

public class HutoolUserAgentUDF extends GenericUDF {
    private transient ObjectInspector[] argumentOIs;
    private transient StructObjectInspector returnOI;

    @Override
    public StructObjectInspector initialize(ObjectInspector[] arguments) {
        this.argumentOIs = arguments;

        ArrayList<String> fieldNames = new ArrayList<>(Arrays.asList("browser",
"browserVersion", "engine", "engineVersion", "os", "osVersion", "platform", "isMobile"));
        ArrayList<ObjectInspector> fieldOIs = new ArrayList<>(Arrays.asList(
            PrimitiveObjectInspectorFactory.javaStringObjectInspector,
            PrimitiveObjectInspectorFactory.javaStringObjectInspector,
            PrimitiveObjectInspectorFactory.javaStringObjectInspector,
            PrimitiveObjectInspectorFactory.javaStringObjectInspector,
            PrimitiveObjectInspectorFactory.javaStringObjectInspector,
            PrimitiveObjectInspectorFactory.javaStringObjectInspector,
            PrimitiveObjectInspectorFactory.javaStringObjectInspector,
            PrimitiveObjectInspectorFactory.javaStringObjectInspector,
            PrimitiveObjectInspectorFactory.javaBooleanObjectInspector
        ));

        returnOI = ObjectInspectorFactory.getStandardStructObjectInspector(fieldNames,
fieldOIs);

        return returnOI;
    }
}

```

```

@Override
public Object evaluate(DeferredObject[] arguments) {
    if (arguments == null || arguments[0] == null) {
        return null;
    }

    Text userAgentString = (Text) arguments[0].get();
    UserAgent userAgent = UserAgentUtil.parse(userAgentString.toString());

    ArrayList<Object> result = new ArrayList<>();
    result.add(userAgent.getBrowser().toString());
    result.add(userAgent.getVersion());
    result.add(userAgent.getEngine().toString());
    result.add(userAgent.getEngineVersion());
    result.add(userAgent.getOs().toString());
    result.add(userAgent.getOsVersion());
    result.add(userAgent.getPlatform().toString());
    result.add(userAgent.isMobile());

    return result;
}

@Override
public String getDisplayString(String[] children) {
    return "parse_user_agent_hutool(" + children[0] + ")";
}
}

```

接下来，编译并打包 Java 项目，将生成的 JAR 文件上传到 Hive 集群。

在 Hive 中创建一个新的临时函数，并指向编译好的 JAR 文件：


```
ADD JAR /path/to/your/compiled/udf.jar;  
CREATE TEMPORARY FUNCTION parse_user_agent_huto
```

反馈

回复没有写完，但是没有必要再使用“继续”了，因为我们已经拿到了函数的实现。现在经过修改的函数已经完全满足我们的需求。

习题三 编写解析 ipv4 地址的 Hive UDF（GPT 做不到）

第 1 轮提示

提示

你知道什么是 ip2region 吗？

答

ip2region 是一个将 IP 地址转换为地理位置信息的工具库。它允许开发者通过 IP 地址查询到相关的国家、省份、城市、运营商等信息。与其他 IP 地址库不同的是，ip2region 使用的是离线数据库，因此不需要进行实时查询，提高了查询速度。ip2region 通常在 IP 地址分析、地理定位、访问统计等场景中使用。

ip2region 的数据来源于开源的 IP 地址库，例如 MaxMind GeoLite2、APNIC、ARIN 和 RIPE 等。数据会定期更新，以保持较高的准确性。ip2region 支持多种编程语言，如 Java、PHP、C#、Python、Node.js 等。

反馈

可以看到 GPT 的知识库里还是有 ip2region 这个开源项目存在的。

第 2 轮提示

提示

请帮我用 java 写一个用 ip2region 查询 114.114.114.114 地理地址信息的 Demo

答

首先，您需要下载 ip2region 库，可以从官方 GitHub 仓库下载：

<https://github.com/lionsoul2014/ip2region>。将 ip2region.jar 包加入到您的 Java 项目中。然后，将 ip2region.db 数据库文件放入您的项目资源文件夹中。

以下是一个简单的 Java 示例，演示了如何使用 ip2region 查询 IP 地址 114.114.114.114 的地理位置信息：

```
import org.lionsoul.ip2region.*;

public class Ip2RegionDemo {
    public static void main(String[] args) {
        // 初始化 ip2region 对象
        String dbPath = "your_project_resources_folder/ip2region.db";
        DbConfig config = null;
        DbSearcher searcher = null;

        try {
            config = new DbConfig();
            searcher = new DbSearcher(config, dbPath);
        } catch (Exception e) {
            System.out.println("Error initializing ip2region: " + e.getMessage());
            return;
        }

        // 查询 IP 地址信息
        String ipAddress = "114.114.114.114";
        DataBlock dataBlock = null;
```

```
try {  
    dataBlock = searcher.memorySearch(ipAddress);  
} catch (Exception e) {  
    System.out.println("Error searching IP: " + e.getMessage());  
    return;  
}  
  
// 输出查询结果  
System.out.println("IP: " + ipAddress);  
System.out.println("Region info: " + dataBlock.getRegion());  
}  
}
```

请注意将 `your_project_resources_folder` 替换为实际的资源文件夹路径。运行此示例后，您应该可以看到 IP 地址 114.114.114.114 的地理位置信息。

提醒：使用 ip2region 查询 IP 地址时，请确保您遵循了相关的法律法规，以及数据隐私和使用政策。

反馈

此处 GPT 算是给了一个旧版实现，但是旧版的数据已经不更新了，这样查询没有意义。

第 3 轮提示

提示

现在 ip2region 已经不提供 db 格式了，请帮我写一个用 xbd 方式查询的代码

答

抱歉刚才的回答给您带来了困扰。由于 ip2region 不再提供 db 格式，我们需要使用 xbd 格式。首先，请从官方 GitHub 仓库下载 ip2region 库：

<https://github.com/lionsoul2014/ip2region>。然后，将 ip2region.jar 包加入到您的 Java 项目中。接下来，将 ip2region.xbd 数据库文件放入您的项目资源文件夹中。

以下是一个简单的 Java 示例，演示了如何使用 ip2region 查询 IP 地址 114.114.114.114 的地理位置信息：

```
import org.lionsoul.ip2region.*;

public class Ip2RegionXbdDemo {
    public static void main(String[] args) {
        // 初始化 ip2region 对象
        String xbdPath = "your_project_resources_folder/ip2region.xbd";
        DbConfig config = null;
        DbSearcher searcher = null;

        try {
            config = new DbConfig();
            searcher = new DbSearcher(config, xbdPath);
        } catch (Exception e) {
            System.out.println("Error initializing ip2region: " + e.getMessage());
            return;
        }

        // 查询 IP 地址信息
        String ipAddress = "114.114.114.114";
        DataBlock dataBlock = null;

        try {
            dataBlock = searcher.memorySearch(ipAddress);
        } catch (Exception e) {
            System.out.println("Error searching IP: " + e.getMessage());
            return;
        }

        // 输出查询结果
        System.out.println("IP: " + ipAddress);
        System.out.println("Region info: " + dataBlock.getRegion());
    }
}
```

```
}
```

反馈

这个时候建议放弃 GPT，立刻转向手动编码，避免浪费时间。